

【分析】

把字母看作结点，单词看成有向边，则问题有解，当且仅当图中有欧拉路径。前面讲过，有向图存在欧拉道路的条件有两个：底图（忽略边方向后得到的无向图）连通，且度数满足上面讨论过的条件。判断连通的方法有两种，一是之前介绍过的 DFS，二是第 11 章中将要介绍的并查集。读者可以在学习完并查集之后根据自己的喜好选用。

6.5 竞赛题目选讲

例题 6-17 看图写树（Undraw the Trees, UVa 10562）

你的任务是将多叉树转化为括号表示法。如图 6-16 所示，每个结点用除了“-”、“|”和空格的其他字符表示，每个非叶结点的正下方总会有一个“|”字符，然后下方是一排“-”字符，恰好覆盖所有子结点的上方。单独的一行“#”为数据结束标记。

样例输入	样例输出
<pre> 2 A --- B C D --- E F G # e --- f g # </pre>	<pre> (A(B()C(E()F())D(G()))) (e(f()g())) </pre>

图 6-16 样例输入与输出

【分析】

直接在二维字符数组里递归即可，无须建树。注意对空树的处理，以及结点标号可以是任意可打印字符。代码如下：

```

#include<cstdio>
#include<cctype>
#include<cstring>
using namespace std;

const int maxn = 200 + 10;
int n;
char buf[maxn][maxn];

//递归遍历并且输出以字符 buf[r][c] 为根的树
void dfs(int r, int c) {
    printf("%c(", buf[r][c]);

```

```

if(r+1 < n && buf[r+1][c] == '|') { //有子树
    int i = c;
    while(i-1 >= 0 && buf[r+2][i-1] == '-') i--; //找“----”的左边界
    while(buf[r+2][i] == '-' && buf[r+3][i] != '\0') {
        if(!isspace(buf[r+3][i])) dfs(r+3, i); //fgets 读入的“\n”也满足 isspace()
        i++;
    }
}
printf(" ");

void solve() {
    n = 0;
    for(;;) {
        fgets(buf[n], maxn, stdin);
        if(buf[n][0] == '#') break; else n++;
    }
    printf("(");
    if(n) {
        for(int i = 0; i < strlen(buf[0]); i++)
            if(buf[0][i] != ' ') { dfs(0, i); break; }
    }
    printf("\n");
}

int main() {
    int T;
    fgets(buf[0], maxn, stdin);
    sscanf(buf[0], "%d", &T);
    while(T--) solve();
    return 0;
}

```

例题 6-18 雕塑 (Sculpture, ACM/ICPC NWERC 2008, UVa12171)

某雕塑由 n ($n \leq 50$) 个边平行于坐标轴的长方体组成。每个长方体用 6 个整数 x_0, y_0, z_0, x, y, z 表示 (均为 1~500 的整数), 其中 x_0 为长方体的顶点中 x 坐标的最小值, x 表示长方体在 x 方向的总长度。其他 4 个值类似定义。你的任务是统计这个雕像的体积和表面积。注意, 雕塑内部可能会有密闭的空间, 其体积应计算在总体积中, 但从“外部”看不见的面不应计入表面积。雕塑可能会由多个连通块组成。

【分析】

设想有一个三维坐标范围均为 1~500 个三维网格, 如果一开始就把输入的 n 个长方体“画”到网格里, 接下来就可以抛开那些长方体, 只在网格中进行统计了。

还记得 floodfill 吗？它不仅能求出连通块的个数，还能准确地找出每个连通块各由哪些方格组成。虽然本题的研究对象是三维空间中的长方体，但丝毫不影响 floodfill 的作用，唯一的区别就是每个格子的相邻格子从二维情形的 4 个增加到了三维情形的 8 个。

本题的麻烦之处在于雕塑中间可能有封闭区域，甚至还有可能相互嵌套，看上去很复杂。但其实可以从反面思考：不考虑雕塑本身，而考虑“空气”。在网格周围加一圈“空气”（目的是为了让所有空气格子连通），然后做一次 floodfill，就可以得到空气的“内表面积”和体积。这个表面积就是雕塑的外表面积，而雕塑体积等于总体积减去空气体积。

但还有一个大问题：空间占用。坐标为 1~500 的整数，一共需要 $500^3=1.25 \times 10^8$ 个单元。在第 5 章的例题“城市正视图”中介绍了离散化法，在这里它再次派上用场：每个维度最多只有 $2n \leq 100$ 个不同的坐标，因此可以把 $500 \times 500 \times 500$ 的网格离散化成 $100 \times 100 \times 100$ ，单元格的数目降为原来的 1/125。在 floodfill 时直接使用离散化后的新网格，但在统计表面积和体积时则需要使用原始坐标。

例题 6-19 自组合 (Self-Assembly, ACM/ICPC World Finals 2013, UVa 1572)

有 n ($n \leq 40000$) 种边上带标号的正方形。每条边上的标号要么为一个写字母后面跟着一个加号或减号，要么为数字 00。当且仅当两条边的字母相同且符号相反时，两条边能拼在一起（00 不能和任何边拼在一起，包括另一条标号为 00 的边）。

假设输入的每种正方形都有无穷多种，而且可以旋转和翻转，你的任务是判断能否组成一个无限大的结构。每条边要么悬空（不和其他边相邻），要么和一个上述可拼接的边相邻。如图 6-17 (a) 所示是 3 个正方形，图 6-17 (b) 所示边是它们组成的一个合法结构（但大小有限）。

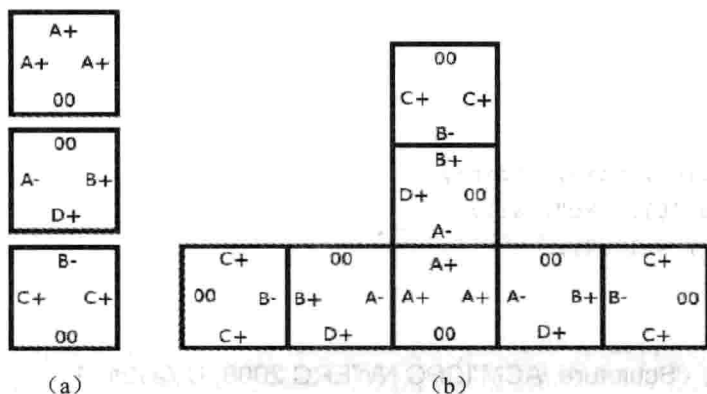


图 6-17 自组合正方形

【分析】

本题看上去很难下手，但不难发现“可以旋转和翻转”是一个很有意思的条件，值得推敲。“无限大结构”并不一定能铺满整个平面，只需要能连出一条无限长的“通路”即可。借助于旋转和翻转，可以让这条“通路”总是往右和往下延伸，因此永远不会自交。这样一来，只需以某个正方形为起点开始“铺路”，一旦可以拼上一块和起点一样的正方形，无限重复下去就能得到一个无限大的结构。

可惜这样的分析仍然不够，因为正方形的数目 n 很大。进一步分析发现：实际上不需要正方形本身重复，而只需要边上的标号重复即可。这样问题就转化为：把标号看成点（一共只有 $A^+ \sim Z^+$, $A^- \sim Z^-$ 这 52 种，因为 00 不能作为拼接点），正方形看作边，得到一个有向图。则当且仅当图中存在有向环时有解。只需要做一次拓扑排序即可。

例题 6-20 理想路径 (Ideal Path, NEERC 2010, UVa1599)

给一个 n 个点 m 条边 ($2 \leq n \leq 100000$, $1 \leq m \leq 200000$) 的无向图，每条边上都涂有一种颜色。求从结点 1 到结点 n 的一条路径，使得经过的边数尽量少，在此前提下，经过边的颜色序列的字典序最小。一对结点间可能有多条边，一条边可能连接两个相同结点。输入保证结点 1 可以达到结点 n 。颜色为 $1 \sim 10^9$ 的整数。

【分析】

首先回顾一下第 3 章中介绍的“字典序”。对于字符串来说，字典序就是在字典里的顺序。例如，ab 在 cd 的前面，cde 在 a 的后面，abcd 在 abcde 的前面。这个定义可以扩展到序列：序列 (1, 2) 在 (3, 4, 5) 的前面，(4, 5, 6) 在 (4, 5) 的后面。

抛开字典序不谈，本题只是一个普通的最短路问题，可以用 BFS 解决。但是之前的“记录父结点”的方法已经不适用了，因为这样打印出来的路径并不能保证字典序最小。怎么办呢？

事实上，无须记录父结点也能得到最短路，方法是从终点开始“倒着”BFS，得到每个结点 i 到终点的最短距离 $d[i]$ ，然后直接从起点开始走，但是每次到达一个新结点时要保证 d 值恰好减少 1（如有多个选择则可以随便走），直到到达终点。可以证明（想一想，为什么）：这样走过的路径一定是一条最短路。

有了上述结论，本题就不难解决了：直接从起点开始按照上述规则走，如果有多种走法，选颜色字典序最小的走；如果有多条边的颜色字典序都是最小，则记录所有这些边的终点，走下一步时要考虑从所有这些点出发的边。聪明的读者应该已经看出来了：这实际上是又做了一次 BFS，因此时间复杂度仍为 $O(m)$ 。其实本题也可以只进行一次 BFS，不过要从终点开始逆向进行，有兴趣的读者可以自行研究。

本题非常重要，强烈建议读者编写程序。

例题 6-21 系统依赖 (System Dependencies, ACM/ICPC World Finals 1997, UVa506)

软件组件之间可能会有依赖关系，例如，TELNET 和 FTP 都依赖于 TCP/IP。你的任务是模拟安装和卸载软件组件的过程。首先是一些 DEPEND 指令，说明软件之间的依赖关系（保证不存在循环依赖），然后是一些 INSTALL、REMOVE 和 LIST 指令，如表 6-1 所示。

表 6-1 指令说明

指 令	说 明
DEPEND item1 item2 [item3 ...]	item1 依赖组件 item2, item3, ...
INSTALL item1	安装 item1 和它的依赖（已安装过的不用重新安装）
REMOVE item1	卸载 item1 和它的依赖（如果某组件还被其他显式安装的组件所依赖，则不能卸载这个组件）
LIST	输出所有已安装组件

在 INSTALL 指令中提到的组件称为显式安装,这些组件必须用 REMOVE 指令显式删除。同样地,被这些显式安装组件所直接或间接依赖的其他组件也不能在 REMOVE 指令中删除。

每行指令包含不超过 80 个字符,所有组件名称都是大小写敏感的。指令名称均为大写字母。

【分析】

这道题目在概念上并没有什么难点,但是有一些细节问题容易写错。首先,维护一个组件名字列表,这样可以把输入中的组件名全部转化为整数编号。接下来用两个 vector 数组 depend[x]和 depend2[x]分别表示组件 x 所依赖的组件列表和依赖于 x 的组件列表(即当读到 DEPEND x y 时要把 y 加入 depend[x],把 x 加入 depend2[y]),这样就可以方便地安装、删除组件,以及判断某个组件是否仍然需要了。

为了区分显式安装和隐式,需要一个数组 status[x],0 表示组件 x 未安装,1 表示隐式显式安装,2 表示隐式安装,则安装组件的代码如下:

```
void install(int item, bool toplevel) {
    if(!status[item]) {
        for(int i = 0; i < depend[item].size(); i++)
            install(depend[item][i], false);
        cout << "    Installing " << name[item] << "\n";
        status[item] = toplevel ? 1 : 2;
        installed.push_back(item);
    }
}
```

删除的顺序相反:首先判断本组件是否能删除,如果可以删除,在删除之后再递归删除它所依赖的组件:

```
bool needed(int item) {
    for(int i = 0; i < depend2[item].size(); i++)
        if(status[depend2[item][i]]) return true;
    return false;
}

void remove(int item, bool toplevel) {
    if((toplevel || status[item] == 2) && !needed(item)) {
        status[item] = 0;
        installed.erase(remove(installed.begin(), installed.end(), item),
                        installed.end());
        cout << "    Removing " << name[item] << "\n";
        for(int i = 0; i < depend[item].size(); i++)
            remove(depend[item][i], false);
    }
}
```